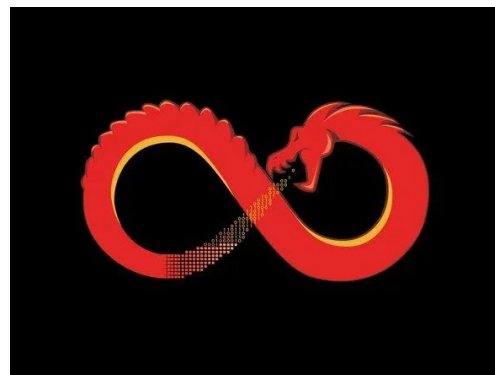
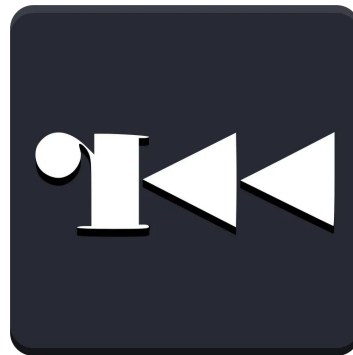

Strumenti principali ed esplorazione software

- Grey & Stefano

Indice strumenti

- Ghidra
- Pwntools (libreria + affiliati)
- Radare2
- Python



1. Introduzione - La trinità di binary exploitation

Per iniziare nel binary exploitation, è fondamentale padroneggiare tre categorie di strumenti principali.

→ **Static Analyzer**

Ghidra, IDA, Binary Ninja.

→ **Dynamic Analyzer**

Debuggers come radare2, GDB, GEF

→ **Exploitation Framework**

Pwntools, che raccoglie un insieme di strumenti con diversi scopi, tra cui la libreria python che useremo

—

**Quali altri strumenti è
importante conoscere prima
del deepdive?**

strings - objdump - readelf

Strings

Strings è uno strumento a linea di comando che consente di mostrare le stringhe all'interno di un file binario.

Strings stampa a schermo stringhe originate da sequenze di minimo 4 caratteri seguiti da un carattere non stampabile.

Se ci sono segreti in chiaro, strings li mostra immediatamente.

Strings

Note importanti:

- -d (o --data): stringhe solo della sezione .data
- -n (o --bytes): minimo numero di caratteri
- -h (o --help): mostra l'aiuto del programma

DIMOSTRAZIONE

Objdump

objdump è uno strumento a linea di comando che consente di analizzare la struttura interna di un file binario.

permette di visualizzare informazioni come le sezioni del programma, i simboli e soprattutto il codice macchina disassemblato.

Objdump

l'opzione '-h' mostra le informazioni principali da ogni header del file:

```
> objdump -h supersecret
```

```
supersecret:      file format elf64-x86-64
```

```
Sections:
```

Idx	Name	Size	VMA	LMA	File off	Algn
0	.interp	0000001c	00000000000000318	00000000000000318	00000318	2**0
	CONTENTS, ALLOC, LOAD, READONLY, DATA					
1	.note.gnu.property	00000030	00000000000000338	00000000000000338	00000338	2**3
	CONTENTS, ALLOC, LOAD, READONLY, DATA					
2	.note.gnu.build-id	00000024	00000000000000368	00000000000000368	00000368	2**2
	CONTENTS, ALLOC, LOAD, READONLY, DATA					
3	.note.ABI-tag	00000020	0000000000000038c	0000000000000038c	0000038c	2**2
	CONTENTS, ALLOC, LOAD, READONLY, DATA					

Objdump

l'opzione '-s' mostra tutte le informazioni delle sezioni. Usato insieme a '-j' consente di selezionare solo una sezione specifica:

```
> objdump -s -j .text supersecret

supersecret:      file format elf64-x86-64

Contents of section .text:
 10e0 f30f1efa 31ed4989 d15e4889 e24883e4 ....1.I..^H..H..
 10f0 f0505445 31c031c9 488d3dca 000000ff .PTE1.1.H.=.....
 1100 15d32e00 00f4662e 0f1f8400 00000000 .....f.....
 1110 488d3df9 2e000048 8d05f22e 00004839 H.=....H.....H9
 1120 f8741548 8b05b62e 00004885 c07409ff .t.H.....H..t..
 1130 e00f1f80 00000000 c30f1f80 00000000 .....
 1140 488d3dc9 2e000048 8d35c22e 00004829 H.=....H.5....H)
 1150 fe4889f0 48c1ee3f 48c1f803 4801c648 .H..H..?H...H..H
```

Objdump

Per concludere , l'opzione '-d' consente di leggere il disassemblato della sezione:

```
> objdump -d -j .text supersecret

supersecret:      file format elf64-x86-64

Disassembly of section .text:

00000000000010e0 <_start>:
10e0:      f3 0f 1e fa      endbr64
10e4:      31 ed           xor     %ebp,%ebp
10e6:      49 89 d1        mov     %rdx,%r9
10e9:      5e             pop     %rsi
10ea:      48 89 e2        mov     %rsp,%rdx
10ed:      48 83 e4 f0     and     $0xfffffffffffffff0,%rsp
10f1:      50             push    %rax
10f2:      54             push    %rsp
10f3:      45 31 c0        xor     %r8d,%r8d
10f6:      31 c9           xor     %ecx,%ecx
10f8:      48 8d 3d ca 00 00 00 lea     0xca(%rip),%rdi      # 11c9 <main>
10ff:      ff 15 d3 2e 00 00 call    *0x2ed3(%rip)       # 3fd8 <__libc_start_main@GLIBC_2.34>
1105:      f4             hlt
1106:      66 2e 0f 1f 84 00 00 cs nopw 0x0(%rax,%rax,1)
110d:      00 00 00
```

Readelf

readelf è un programma a linea di comando
SPECIALIZZATO nell'analisi dei file con struttura ELF.

permette di ispezionare un eseguibile o libreria per
comprenderne la struttura interna senza eseguire lo stesso.

Readelf

L'opzione '-h' stampa le informazioni dell'header:

```
> readelf -h supersecret
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x10e0
  Start of program headers:              64 (bytes into file)
  Start of section headers:              14168 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              13
  Size of section headers:               64 (bytes)
  Number of section headers:              31
  Section header string table index:     30
```

Readelf

L'opzione '-l' permette di mappare le varie sezioni:

```
Section to Segment mapping:
Segment Sections...
00
01      .interp
02      .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .gnu.hash .dynsym .dynstr .gnu.version .gnu.version
n_r .rela.dyn .rela.plt
03      .init .plt .plt.got .plt.sec .text .fini
04      .rodata .eh_frame_hdr .eh_frame
05      .init_array .fini_array .dynamic .got .data .bss
06      .dynamic
07      .note.gnu.property
08      .note.gnu.build-id .note.ABI-tag
09      .note.gnu.property
10      .eh_frame_hdr
11
12      .init_array .fini_array .dynamic .got
```

Readelf

Per concludere , l'opzione '-s' stampa le entries nella sezione della symbol table:

```
> readelf -s supersecret
```

Symbol table '.dynsym' contains 11 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	_[...]@GLIBC_2.34 (2)
2:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__ITM_deregisterT[...]
3:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	puts@GLIBC_2.2.5 (3)
4:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	__[...]@GLIBC_2.4 (4)
5:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@GLIBC_2.2.5 (3)
6:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	[...]@GLIBC_2.2.5 (3)
7:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__gmon_start__
8:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	__[...]@GLIBC_2.7 (5)
9:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	__ITM_registerTMC[...]
10:	0000000000000000	0	FUNC	WEAK	DEFAULT	UND	[...]@GLIBC_2.2.5 (3)

Symbol table '.symtab' contains 40 entries:

```

[0x5b318a6ef1c9 [xaDvc]0 150 /home/grey/cyberchallenge-binary/lezione
stopped at 0x00000000
- offset -      C0C1 C2C3 C4C5 C6C7 C8C9 CACB CCCD CECF 0123456789AB
0x7ffd11bde1c0 0100 0000 0000 0000 dde4 bd11 fd7f 0000 .....
0x7ffd11bde1d0 0000 0000 0000 0000 ebe4 bd11 fd7f 0000 .....
0x7ffd11bde1e0 f5e4 bd11 fd7f 0000 02e5 bd11 fd7f 0000 .....
0x7ffd11bde1f0 12e5 bd11 fd7f 0000 90e5 bd11 fd7f 0000 .....
s:0 z:0 c:0 o:0 p:0
rax 0x00000000      rbx 0x00000000      rcx 0x00000000
rdx 0x00000000      r8 0x00000000      r9 0x00000000
r10 0x00000000     r11 0x00000000     r12 0x00000000
r13 0x00000000     r14 0x00000000     r15 0x00000000
rsi 0x00000000     rdi 0x00000000     rsp 0x7ffd11bde1
rbp 0x00000000     rip 0x7e9b71f86540   rflags 0x00000200
orax 0x0000003b
;[x] DATA XREF from entry0 @ 0x5b318a6ef0f8(r)
159: int main (int argc, char **argv, char **envp);
afv: vars(2:sp[0x10..0x78])
0x5b318a6ef1c9      f30f1efa      endbr64
0x5b318a6ef1cd      55           push rbp
0x5b318a6ef1ce      4889e5       mov rbp, rsp
0x5b318a6ef1d1      4883ec70     sub rsp, 0x70
0x5b318a6ef1d5      64488b0425.. mov rax, qword fs:[0x2
0x5b318a6ef1de      488945f8     mov qword [var_8h], ra
0x5b318a6ef1e2      31c0        xor eax, eax
0x5b318a6ef1e4      488d051d0e.. lea rax, str.Aldo_ha_d
0x5b318a6ef1eb      4889c7       mov rdi, rax
0x5b318a6ef1ee      b800000000   mov eax, 0
0x5b318a6ef1f3      e8b8feffff   call sym.imp.printf
0x5b318a6ef1f8      488d4590     lea rax, [var_70h]

```

Radare2

è un framework open-source per reverse engineering e analisi di binary, in grado di utilizzare internamente ghidra.

Come iniziare ad usarlo:

Prima di tutto, bisogna aprire il programma con r2 ed indicare che si vuole effettuare il debugging:

`r2 -d nomebinary`

```
> r2 -d supersecret  
-- I nodejs so hard my exams.  
[0x74de2ef7e540]>
```

Analizzare i binari:

Successivamente, bisogna dire a r2 che tipo di analisi vogliamo effettuare inserendo:

- a -> analisi semplice
- aa -> analizza tutto
- aaa -> analisi profonda con euristiche

```
[0x74de2ef7e540]> a
fcns      0
xrefs     0
calls     0
strings   5
symbols   13
imports   5
coverage  1208
codesz    -1
percent   0%
```

```
[0x74de2ef7e540]> aa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Recovering variables (afva@@F)
INFO: Analyze all functions arguments/locals (afva@@F)
[0x74de2ef7e540]> |
```

Come aiutarsi in caso di problemi:

per qualsiasi dubbio, r2 permette di ricercare cosa fa il comando, usando il carattere '?' a successione di un comando

```
[0x74de2ef7e540]> a?  
Usage: a [abdefFghoprxtc] [...]  
| a          alias for aai - analysis information  
| a:[cmd]    run a command implemented by an analysis plugin (like : for io)  
| a*        same as afl*;ah*;ax*  
| aa[?]     analyze all (fcns + bbs) (aa0 to avoid sub renaming)
```

Come vedere le funzioni nel codice:

Una volta analizzato il codice, conviene sempre avere una panoramica delle funzioni presenti:

afl -> analyze function list

```
[0x7f892c4f0540]> afl
0x585bfad43040 1 38 entry0
0x585bfad45fd8 1 4137 fcn.585bfad45fd8
0x585bfad43070 4 34 sym.deregister_tm_clones
0x585bfad430a0 4 51 sym.register_tm_clones
0x585bfad430e0 5 54 entry.fini0
0x585bfad43030 1 10 fcn.585bfad43030
0x585bfad43120 1 9 entry.init0
0x585bfad43165 1 15 sym.fun2
0x585bfad43184 1 13 sym._fini
0x585bfad43156 1 15 sym.fun1
0x585bfad43174 1 15 sym.fun3
0x585bfad43129 1 45 main
0x585bfad43000 3 27 sym._init
```

Come facilitare l'uso:

Viene comodo avere un interfaccia grafica. Quindi usiamo il comando per evocare una interfaccia testuale avanzata simile ad una GUI

Vpp -> VisualMode

* p viene usato per scorrere tra le varie interfacce disponibili, quella utile sta dopo 2 scorrimenti, quindi Vpp

esempio ingrandito a pagina succ.

```
[0x585bfad43129 [xaDvc]0 150 /home/grey/cyberchallenge-binary/lezione0/multifunc/multi] diq;?t0;f .. @ sym.main
stopped at 0x00000000
- offset -      E0E1 E2E3 E4E5 E6E7 E8E9 EAEB ECED EEEF 0123456789ABCDEF
0x7ffd23651ee0 0100 0000 0000 0000 e724 6523 fd7f 0000 .....$e#....
0x7ffd23651ef0 0000 0000 0000 0000 ef24 6523 fd7f 0000 .....$e#....
0x7ffd23651f00 f924 6523 fd7f 0000 0625 6523 fd7f 0000 ..$e#.....$e#....
0x7ffd23651f10 1625 6523 fd7f 0000 9425 6523 fd7f 0000 ..$e#.....$e#....
s:0 z:0 c:0 o:0 p:0
rax 0x00000000      rbx 0x00000000      rcx 0x00000000
rdx 0x00000000      r8 0x00000000      r9 0x00000000
r10 0x00000000     r11 0x00000000     r12 0x00000000
r13 0x00000000     r14 0x00000000     r15 0x00000000
rsi 0x00000000     rdi 0x00000000     rsp 0x7ffd23651ee0
rbp 0x00000000     rip 0x7f892c4f0540  rflags 0x00000200
orax 0x00000003b
;[x] DATA XREF from entry0 @ 0x585bfad43058(r)
45: int main (int argc, char **argv, char **envp);
      0x585bfad43129 f30f1efa endbr64
      0x585bfad4312d 55      push rbp
      0x585bfad4312e 4889e5   mov rbp, rsp
      0x585bfad43131 b800000000 mov eax, 0
      0x585bfad43136 e81b000000 call sym.fun1 ;[1]
      0x585bfad4313b b800000000 mov eax, 0
      0x585bfad43140 e820000000 call sym.fun2 ;[2]
      0x585bfad43145 b800000000 mov eax, 0
      0x585bfad4314a e825000000 call sym.fun3 ;[3]
      0x585bfad4314f b800000000 mov eax, 0
      0x585bfad43154 5d      pop rbp
      0x585bfad43155 c3      ret
```

Guardare alle funzioni:

In visual mode, per eseguire i comandi, bisogna inserire ":" per evocare il terminale. Per saltare ad una funzione, si usa il comando "s":

s nomefun -> seek function

Esempio seek + vpp main

```
[0x585bfad43129 [xaDvc]0 150 /home/grey/cyberchallenge-binary/lezione0/multifunc/multi]> diq;?t0;f .. @ sym.main
stopped at 0x00000000
```

- offset -	E0E1	E2E3	E4E5	E6E7	E8E9	EAEB	ECED	EEEF	0123456789ABCDEF
0x7ffd23651ee0	0100	0000	0000	0000	e724	6523	fd7f	0000	\$e#....
0x7ffd23651ef0	0000	0000	0000	0000	ef24	6523	fd7f	0000	\$e#....
0x7ffd23651f00	f924	6523	fd7f	0000	0625	6523	fd7f	0000	.\$e#.....%e#....
0x7ffd23651f10	1625	6523	fd7f	0000	9425	6523	fd7f	0000	.%e#.....%e#....

s:0 z:0 c:0 o:0 p:0

rax	0x00000000	rbx	0x00000000	rcx	0x00000000
rdx	0x00000000	r8	0x00000000	r9	0x00000000
r10	0x00000000	r11	0x00000000	r12	0x00000000
r13	0x00000000	r14	0x00000000	r15	0x00000000
rsi	0x00000000	rdi	0x00000000	rsp	0x7ffd23651ee0
rbp	0x00000000	rip	0x7f892c4f0540	rflags	0x00000200
orax	0x0000003b				

;[x] DATA XREF from entry0 @ 0x585bfad43058(r)

45: int main(int argc, char **argv, char **envp);

0x585bfad43129	f30f1efa	endbr64	
0x585bfad4312d	55	push rbp	
0x585bfad4312e	4889e5	mov rbp, rsp	
0x585bfad43131	b800000000	mov eax, 0	
0x585bfad43136	e81b000000	call sym.fun1	;[1]
0x585bfad4313b	b800000000	mov eax, 0	
0x585bfad43140	e820000000	call sym.fun2	;[2]
0x585bfad43145	b800000000	mov eax, 0	
0x585bfad4314a	e825000000	call sym.fun3	;[3]
0x585bfad4314f	b800000000	mov eax, 0	
0x585bfad43154	5d	pop rbp	
0x585bfad43155	c3	ret	

Iniziare a settare il debugging:

Per mettere/togliere un breakpoint, ci sono due maniere:

**db address / db nomefun+offset
oppure
muovendosi con le frecce in Vpp
con maiusc+b**

**premendo 'c' in visual mode si cambia la
modalità di visualizzazione sullo stack**

dimostrazione pratica

Comandi debugger:

Per iniziare a debuggare, si usano questi comandi:

- **ds** -> debugger step
- **dc** -> debugger continue
- **dso** -> debugger step out
- **dsi** -> debugger step in
- **db** -> lista di breakpoints

dimostrazione pratica

In caso di problemi come riavviare:

Per ricominciare l'esecuzione, si usa il comando "ood"

- **ood -> reopen in debug mode**

dimostrazione pratica

Interazione con i registri:

Per interagire con i registri, usiamo il comando “dr”:

- **dr-> debugger register**
- **dr rax -> stampa contenuto di rax**
- **dr rax=0x100 -> setta il valore di rax**

dimostrazione pratica

Come sbirciare nella memoria:

Per concludere, per interagire con la memoria, possiamo usare i comandi w e p:

- **p @ address -> peek at address**
- **w value @ add-> write bytes at**

Di fianco a 'p' e 'w' possono essere usati dei formattatori, che indicano come trattare i dati:

- **px @ address -> peek at address as hex**
- **wx value @ add-> write hex at addr**

dimostrazione pratica